

ASSIGNMENT-2

M.SaiRam

[Reg.No](#) : 192425291

PROBLEM 1: REQUIREMENTS ELICITATION & PRIORITIZATION IN A COLLEGE ATTENDANCE SYSTEM

1.1 Introduction

A college attendance management system is used to record, maintain, and monitor student attendance. Different stakeholders have different expectations from the system. Students want easy access to their attendance, teachers want accurate attendance recording and biometric integration, while management wants a low-cost system that can be developed and delivered quickly.

Requirements Engineering techniques such as requirements elicitation, MoSCoW prioritization, and the Kano Model can be used to identify and prioritize these requirements.

1.2 Four Well-Defined Requirements

Requirement 1: App-Based Attendance Checking

The system shall allow students to securely view their attendance percentage and subject-wise attendance through a mobile or web application.

Requirement 2: Attendance Recording

The system shall allow teachers to record and update student attendance for each class.

Requirement 3: Biometric Integration

The system shall support integration with biometric devices to automatically record student attendance.

Requirement 4: Low-Cost and Quick Deployment

The system shall be developed using cost-effective technologies and deployed within a short development schedule.

1.3 MoSCoW Prioritization

MoSCoW prioritization divides requirements into four categories: Must Have, Should Have, Could Have, and Won't Have.

Requirement 1 – App-Based Attendance Checking

Priority: MUST HAVE

Reason: Students need convenient and quick access to their attendance information.

Requirement 2 – Attendance Recording

Priority: MUST HAVE

Reason: Recording attendance is the main function of the attendance management system.

Requirement 3 – Biometric Integration

Priority: SHOULD HAVE

Reason: Biometric integration improves attendance accuracy and reduces manual work, but it can be implemented in a later phase if budget or time is limited.

Requirement 4 – Low-Cost and Quick Deployment

Priority: MUST HAVE

Reason: Management has specifically requested a low-cost solution that can be delivered quickly.

Could Have Features:

Advanced attendance analytics, graphical reports, automatic notifications, and attendance prediction can be added in future releases.

Won't Have in the First Release:

Advanced artificial intelligence-based attendance prediction and facial recognition can be excluded from the first release because they increase development cost and complexity.

1.4 MoSCoW Prioritization Table

Requirement Priority Reason

App-Based Attendance Checking MUST HAVE Essential for students to check attendance.

Attendance Recording MUST HAVE Core function of the system.

Biometric Integration SHOULD HAVE Improves accuracy but can be added later.

Low-Cost Quick Deployment MUST HAVE Required by management.

1.5 Kano Model Classification

Basic Features:

Attendance recording is a Basic feature because users expect the system to record attendance correctly. If this feature is missing or unreliable, users will be dissatisfied.

Performance Features:

Student attendance checking is a Performance feature. As the system becomes faster, more accurate, and easier to use, user satisfaction increases.

Excitement Features:

Biometric integration can be considered an Excitement feature. Users may not initially expect it, but its availability can provide additional convenience and improve satisfaction.

1.6 Kano Classification Table

Feature	Kano Category	Explanation
Attendance Recording	Basic	Expected core functionality.
Attendance Checking Performance	Performance	Better speed and accuracy increase satisfaction.
Biometric Integration	Excitement	Provides additional convenience and automation.

1.7 Stakeholder Conflict

The main conflict is between students, teachers, and management.

Students want a mobile application for checking attendance. Teachers want biometric integration because it can reduce manual attendance work and improve accuracy. However, management wants the system to be inexpensive and delivered quickly. Biometric hardware and integration may increase both development cost and implementation time.

1.8 Conflict Resolution Plan

1. Conduct a stakeholder meeting to understand the importance and urgency of each requirement.
2. Use MoSCoW prioritization to identify the essential features for the first release.
3. Develop attendance recording and attendance checking as the first version.
4. Implement biometric integration as a second-phase feature.
5. Review the cost, schedule, and technical feasibility with all stakeholders.
6. Obtain stakeholder agreement before finalizing the requirements.

Conclusion:

MoSCoW prioritization helps the team focus on the most important requirements, while the Kano Model helps understand how different features affect user satisfaction. A phased implementation approach can satisfy stakeholder needs without exceeding the project budget or schedule.

PROBLEM 2: IMPROVING COLLABORATION WITH JIRA & CONFLUENCE IN A HOSPITAL SYSTEM

2.1 Introduction

A hospital appointment booking system requires close collaboration between developers, designers, testers, business analysts, and other stakeholders. If developers use Jira for requirements and designers independently use Confluence for documentation, information can become inconsistent or duplicated. Proper integration between Jira and Confluence can provide a common source of information and improve team collaboration.

2.2 Conflict Type Identification

The main conflict is a Communication and Process Conflict.

The problem occurs because different teams use different tools and maintain information separately. This can result in:

1. Duplicate requirements.
2. Missing updates.
3. Outdated information.
4. Confusion about responsibilities.
5. Repeated development work.
6. Delays in completing tasks.
7. Blaming between teams because of unclear communication.

2.3 Jira and Confluence Integration

Jira should primarily be used for project and task management, while Confluence should be used for detailed documentation and knowledge management.

Jira can contain:

- User stories
- Tasks
- Bugs
- Sprint information
- Priorities
- Assignees
- Development status

Confluence can contain:

- Detailed requirements
- Design documents
- Architecture diagrams
- Meeting notes
- Business rules
- Technical documentation

Jira issues can contain links to the relevant Confluence pages. Similarly, Confluence pages can contain links to related Jira tasks.

This creates traceability between requirements, documentation, development, and testing.

2.4 Example

Confluence Requirement:

"The hospital appointment system shall allow patients to search for doctors based on specialization and availability."

Related Jira Issue:

HOSP-101 – Implement Doctor Search

The Jira issue contains a link to the Confluence requirement page. Developers can therefore understand exactly which requirement they are implementing.

2.5 Conflict Resolution Step 1 – Establish a Single Source of Truth

The team should agree that approved requirements and detailed documentation are maintained in Confluence, while Jira contains the implementation tasks created from those requirements.

This prevents multiple versions of the same requirement from being maintained in different locations.

2.6 Conflict Resolution Step 2 – Regular Synchronization

The team should conduct regular requirement review meetings, sprint planning sessions, and sprint reviews.

Whenever a requirement changes, the corresponding Jira task and Confluence documentation should be updated.

This ensures that developers, designers, testers, and managers are working with the latest information.

2.7 Workflow for Linking Tasks and Documents

The following workflow can be used:

Business Requirement



Confluence Requirement Page



Create Jira User Story



Jira Task / Sub-task



Development



Testing



Jira Status Updated



Confluence Documentation Updated



Sprint Review

Example:

Confluence:

"Doctor Appointment Requirement"



Jira:

"HOSP-101 – Implement Doctor Search"



Development → Testing → Review



Jira Issue Marked as DONE



Confluence Documentation Updated

Conclusion:

Integrating Jira and Confluence improves communication, reduces duplicated work, and provides traceability between requirements and development tasks. It also reduces misunderstandings between developers and designers.

PROBLEM 3: CHOOSING ARCHITECTURE FOR A SCALABLE FOOD DELIVERY PLATFORM

3.1 Introduction

A food delivery platform similar to Swiggy or Zomato must handle thousands of customers, restaurants, delivery partners, orders, payments, and notifications. During peak hours, the system may experience a very large number of simultaneous requests.

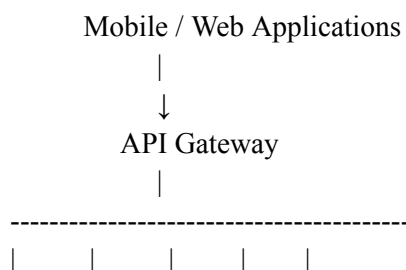
Therefore, the architecture must support scalability, reliability, fault isolation, and independent deployment.

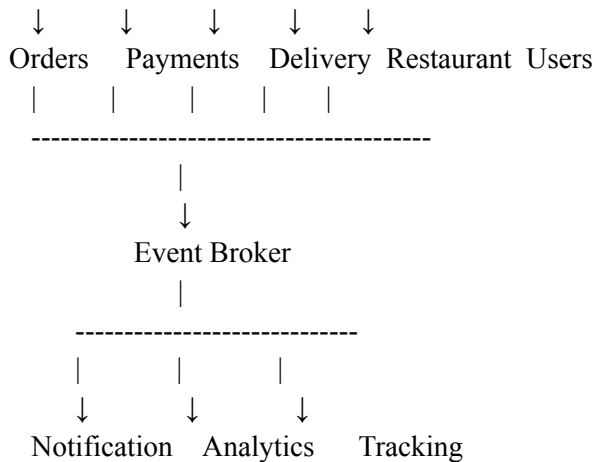
3.2 Recommended Architecture

The recommended architecture is Microservices Architecture combined with Event-Driven Architecture.

In this approach, the application is divided into multiple independent services. Each service is responsible for a specific business function.

Basic Architecture:





3.3 Technical Reason 1 – Independent Scalability

Different services receive different amounts of traffic.

For example, during lunch and dinner hours, the Order Service may receive thousands of requests per minute. With microservices, the Order Service can be scaled independently without scaling every other service.

This improves resource utilization and reduces cost.

3.4 Technical Reason 2 – Fault Isolation

Microservices provide better fault isolation.

For example, if the Notification Service becomes unavailable, customers may not receive notifications, but the Order Service and Payment Service can continue processing orders.

Therefore, failure in one service does not necessarily bring down the entire application.

3.5 Technical Reason 3 – Independent Development and Deployment

Different development teams can work on different services independently.

For example, the payment team can modify the Payment Service without redeploying the complete food delivery application.

This makes development faster and supports continuous delivery.

3.6 Event-Driven Architecture

Event-driven architecture allows services to communicate using events.

For example, when a customer places an order:

Customer Places Order



Order Service



"OrderPlaced" Event



Event Broker



Payment Restaurant Delivery

Service Service Service

The Order Service publishes an "OrderPlaced" event. Other services can independently respond to this event.

The Payment Service processes the payment.

The Restaurant Service receives the order.

The Delivery Service searches for a delivery partner.

The Notification Service sends a confirmation to the customer.

Benefits of Event-Driven Architecture:

1. Loose coupling between services.
2. Better scalability.
3. Asynchronous processing.
4. Improved fault tolerance.
5. Faster communication between independent services.
6. Easy addition of new services.

3.7 Suitable Microservices

1. Order Service

Responsibilities:

- Create orders
- Update order status
- Cancel orders
- Maintain order history

2. Payment Service

Responsibilities:

- Process payments
- Verify transactions
- Process refunds
- Maintain payment status

3. Delivery Service

Responsibilities:

- Assign delivery partners
- Track delivery
- Calculate estimated delivery time
- Update delivery status

Other possible services include User Service, Restaurant Service, Menu Service, Notification Service, Location Service, and Recommendation Service.

Conclusion:

Microservices combined with event-driven architecture is suitable for a large-scale food delivery platform because it provides scalability, fault isolation, independent deployment, and efficient asynchronous communication.

PROBLEM 4: API-FIRST DEVELOPMENT FOR A HOTEL BOOKING PLATFORM

4.1 Introduction

A hotel booking platform may have multiple development teams working on web applications, mobile applications, backend services, and testing. API-first development allows all teams to work using a common API contract.

OpenAPI/Swagger can be used to define and document the API before implementation.

4.2 Advantages of API-First Development

API-first development is suitable because:

1. Frontend and backend teams can work in parallel.
2. Mobile and web developers have a common API specification.
3. Integration problems can be identified early.
4. APIs are clearly documented.
5. Mock APIs can be created before backend implementation is completed.
6. Testing becomes easier.
7. The same API can support multiple client applications.

4.3 Swagger-Style API – Room Availability Check

Endpoint:

GET /api/v1/hotels/{hotelId}/rooms/availability

Purpose:

This API checks whether rooms are available for a particular hotel and date range.

Parameters:

hotelId – String – Required – Unique hotel identifier.

checkIn – Date – Required – Customer check-in date.

checkOut – Date – Required – Customer check-out date.

guests – Integer – Required – Number of guests.

Example Request:

GET /api/v1/hotels/H1001/rooms/availability?checkIn=2026-09-10&checkOut=2026-09-12&guests=2

Example Response:

```
{  
  "hotelId": "H1001",  
  "checkIn": "2026-09-10",
```

```
"checkOut": "2026-09-12",  
"availableRooms": [  
  {  
    "roomId": "R101",  
    "roomType": "Deluxe",  
    "pricePerNight": 4500,  
    "currency": "INR"  
  }  
]  
}
```

Swagger/OpenAPI Outline:

paths:

/api/v1/hotels/{hotelId}/rooms/availability:

get:

summary: Check room availability

parameters:

hotelId:

type: string
required: true

checkIn:

type: string
format: date
required: true

checkOut:

type: string
format: date
required: true

guests:

type: integer
required: true

responses:

200:

description: Available rooms returned successfully

400:

description: Invalid request

404:

description: Hotel not found

4.4 API Versioning Strategy

URL-based versioning can be used.

Version 1:

/api/v1/hotels/...

If a major breaking change is introduced, Version 2 can be created:

/api/v2/hotels/...

The existing v1 API should continue to support current applications while clients are gradually migrated to v2.

The following practices should be followed:

1. Avoid unnecessary breaking changes.
2. Maintain older versions for existing clients.
3. Provide migration documentation.
4. Inform users before retiring an old API version.
5. Introduce a new version only when significant changes are required.

4.5 Benefits of Docker Containerization

Benefit 1 – Consistent Environment

Docker packages the API application, libraries, dependencies, and configuration into a container. This ensures that the API behaves consistently across development, testing, and production environments.

Benefit 2 – Easy Deployment and Scaling

Docker containers can be quickly created, deployed, replicated, and scaled. Multiple API containers can therefore handle increased hotel booking traffic.

Conclusion:

API-first development provides a common contract for frontend, backend, and mobile teams. OpenAPI/Swagger improves API documentation and collaboration, while Docker provides consistent and scalable deployment.

PROBLEM 5: ENSURING RELIABILITY USING FORMAL VERIFICATION & DESIGN BY CONTRACT

5.1 Introduction

A banking payment system must be highly reliable because even a small software error can result in financial losses. Formal verification and Design by Contract can be used to specify and verify important system properties.

Formal verification uses mathematical methods to prove that software satisfies specific requirements.

5.2 Formal Verification Use-Case

Formal verification can be used to verify the fund transfer operation of a banking system.

The system should guarantee that:

1. A transfer cannot be made from a non-existent account.
2. The transfer amount must be greater than zero.
3. The sender must have sufficient funds.
4. The sender's balance is correctly reduced.
5. The receiver's balance is correctly increased.
6. The same transaction cannot be processed twice.
7. Money cannot be accidentally created or lost during the transfer.

Example:

Before transfer:

Sender Balance = ₹10,000

Transfer Amount = ₹3,000

After successful transfer:

Sender Balance = ₹7,000

Receiver Balance = Previous Receiver Balance + ₹3,000

Formal verification can help prove that the required properties are maintained during the transaction.

5.3 Design by Contract for Fund Transfer

Design by Contract defines three important elements:

1. Preconditions
2. Postconditions
3. Invariants

5.3.1 Preconditions

The following conditions must be true before a fund transfer starts:

P1: Sender account must exist.

P2: Receiver account must exist.

P3: Sender account must be active.

P4: Receiver account must be active.

P5: Transfer amount must be greater than zero.

P6: Sender must have sufficient balance.

P7: Sender and receiver must be valid accounts.

5.3.2 Postconditions

After a successful transfer:

Q1: $\text{Sender balance} = \text{Old sender balance} - \text{Transfer amount}$.

Q2: $\text{Receiver balance} = \text{Old receiver balance} + \text{Transfer amount}$.

Q3: A unique transaction record must be created.

Q4: Transaction status must be SUCCESS.

Q5: The transfer amount must not be lost or duplicated.

5.3.3 Invariants

The following conditions must always remain true:

I1: Account balance cannot become negative unless overdraft is explicitly allowed.

I2: Every transaction must have a unique transaction ID.

I3: A completed transaction cannot be processed again.

I4: Account balances must remain consistent with transaction records.

5.4 Simplified Design by Contract Specification

FundTransfer(Sender, Receiver, Amount)

Preconditions:

Sender exists

Receiver exists

Sender is active

Receiver is active

$\text{Amount} > 0$

$\text{Sender.balance} \geq \text{Amount}$

Postconditions:

$\text{Sender.balance} = \text{old}(\text{Sender.balance}) - \text{Amount}$

$\text{Receiver.balance} = \text{old}(\text{Receiver.balance}) + \text{Amount}$

$\text{Transaction.status} = \text{SUCCESS}$

Transaction ID is unique

Invariants:

Transaction IDs are unique.

Completed transactions cannot be duplicated.

Account balances remain consistent with transaction records.

5.5 How Contracts Improve System Reliability

Design by Contract improves reliability in several ways.

1. Early Error Detection

Invalid input can be rejected before the transaction is processed.

2. Reduced Ambiguity

The contract clearly defines what the system expects and what it must produce.

3. Better Testing

Preconditions and postconditions can be directly converted into test cases.

4. Prevention of Invalid States

Invariants ensure that important system rules remain true throughout the execution of the system.

5. Easier Maintenance

Developers can understand the expected behavior of the system from the contract.

6. Financial Protection

Incorrect or invalid transactions can be detected before they result in financial loss.

5.6 Real-World Failure Example

A well-known example is the Knight Capital Group trading incident in 2012. A software deployment problem caused an automated trading system to execute a large number of unintended trades. The company suffered losses of approximately 440 million US dollars in around 45 minutes.

This incident demonstrates the importance of proper software testing, verification, controlled deployment, and safety mechanisms in financial systems.

A combination of formal specifications, extensive testing, deployment controls, and runtime safeguards could help identify or limit similar failures.

CONCLUSION

Requirements Engineering and Software Architecture are important for developing high-quality, reliable, and scalable software systems.

In the college attendance system, requirements elicitation, MoSCoW prioritization, and the Kano Model help manage conflicting stakeholder requirements.

In the hospital appointment system, Jira and Confluence integration improves communication, documentation, and traceability.

For a large food delivery platform, microservices combined with event-driven architecture provides scalability, fault isolation, and independent deployment.

For the hotel booking platform, API-first development allows frontend, backend, and mobile teams to work in parallel using a common API contract. Docker further improves consistency and deployment.

Finally, formal verification and Design by Contract provide strong techniques for improving reliability in banking and other high-risk systems.

These techniques help organizations develop software that is efficient, maintainable, reliable, and aligned with stakeholder requirements.